Software Design Patterns for Temporal Upscaling Compatibility and Performance in Real-Time Games

Owen Newberry and Samuel Sungmin Cho

Northern Kentucky University
{newberryo1, chos5}@nku.edu

***Abstract*—Modern real-time rendering increasingly relies on temporal upscaling and AI-assisted frame generation techniques to achieve high visual fidelity at sustainable performance levels. However, the effectiveness of these techniques depends critically on how game software is structured. This research identifies the software-side constraints imposed by temporal upscaling methods such as FSR 2/3, DLSS, and engine-native temporal super resolution and proposes four concrete design patterns that help game developers produce the stable motion data, predictable timing, and clean rendering inputs these systems require. Through analysis of vendor documentation, engine implementations, and established game-programming patterns, this work bridges the gap between graphics-level upscaler requirements and practical software engineering guidance. A prototype implementation demonstrates how these patterns—Fixed-Timestep Simulation (including fixed-rate integration and, in the full formulation, optional render interpolation), Single-Writer Motion Authority, Time-Based Effects and Animations, and Bounded Workload Budgeting—each address a specific temporal failure mode, with empirical runs showing the intended simulation-side effect for the oscillator, single-writer policy, time-driven motion, and workload-budget conditions tested. The findings provide actionable design guidance for student and indie developers working at the intersection of software architecture and modern GPU-accelerated rendering.**

I. Introduction

Over the past two decades, video games have evolved into complex, real-time systems that must balance visual fidelity, responsiveness, and performance across a wide range of hardware. Modern rendering pipelines increasingly rely on temporal techniques—temporal antialiasing (TAA), temporal upscaling, and frame interpolation—to deliver high perceived quality at feasible performance costs, especially at 4K resolution and beyond. These methods reconstruct detail by combining information from multiple frames, guided by engine-supplied data such as motion vectors, depth buffers, and jittered camera samples.

At the same time, graphics processing units (GPUs) have shifted from fixed-function graphics devices toward highly parallel, AI-capable processors. AI-assisted upscalers and frame-generation systems such as NVIDIA's Deep Learning Super Sampling and AMD's FidelityFX Super Resolution now form a core part of many rendering pipelines, and next-generation architectures like NVIDIA's Ada Lovelace and Blackwell families are explicitly designed with these workloads in mind. Features such as real-time ray tracing, neural upscaling, frame generation, and AI-assisted animation systems are no longer experimental add-ons—they are becoming core assumptions in modern game engines.

However, the effectiveness of these temporal and AI-assisted techniques is limited not only by GPU hardware but also by how game software is structured. Design choices in game loops, state management, animation systems, and effects can either provide clean, stable inputs to the upscaler—or introduce inconsistencies that lead to ghosting, smearing, flicker, and unstable frame pacing. Student and indie projects, which often lack the infrastructure of large commercial engines, are particularly vulnerable to these issues.

Frame-rate-coupled game logic, inconsistent motion vectors, multiple systems fighting over transform data, and frame-based rather than time-based effects can all break temporal reconstruction, resulting in visible artifacts and degraded performance. This project investigates how software design patterns for real-time game systems can be structured to better support temporal upscaling and frame generation.

Rather than focusing on the internal implementation of upscalers themselves, this research approaches temporal upscaling as a set of constraints and assumptions placed on the game's code: predictable timing, coherent motion data, and well-defined ownership of transforms and effects. The goal is to identify, implement, and evaluate practical patterns that student and indie developers can adopt to avoid breaking temporal reconstruction, thereby improving both visual quality and performance.

This topic connects directly to Applied Software Engineering and the principles of performance-oriented architecture. It sits at the intersection of real-time systems design, performance engineering, and graphics-aware software architecture, asking: how do classic software-engineering concerns—such as fixed-timestep loops, task scheduling, and ownership patterns—need to evolve in the era of temporal and AI-assisted rendering?

A. Research Question

How can software design patterns for real-time game systems be structured to produce stable motion data, predictable timing, and clean rendering inputs that improve the effectiveness of temporal upscaling and frame-generation techniques in modern games?

II. Background and Literature Review

A. Temporal Upscaling and Frame Generation

Temporal upscaling techniques reconstruct high-resolution images from lower-resolution inputs by combining data across multiple frames. Unlike spatial upscaling methods that operate on a single frame, temporal approaches leverage motion vectors, depth information, and camera jitter to reproject and accumulate samples over time. This temporal accumulation allows the system to borrow detail from previous frames, effectively amortizing rendering cost across time rather than space.

AMD's FidelityFX Super Resolution 2 (FSR2) represents a widely-adopted temporal upscaling solution. According to AMD's GDC 2023 presentation on temporal upscaling [1], FSR2 requires several key inputs from the game engine: a low-resolution color buffer, depth buffer, per-pixel motion vectors in screen space, camera jitter offsets, and optional reactive/transparency-and-composition masks for handling special cases like particles and animated textures. The quality of these inputs directly determines the effectiveness of the upscaler: poor motion vectors lead to ghosting, incorrect depth causes disocclusion artifacts, and missing jitter information results in flickering.

NVIDIA's Deep Learning Super Sampling (DLSS) follows similar principles but incorporates neural network inference to reconstruct detail. The DLSS programming guide [2] specifies that games must provide motion vectors, depth, exposure, and jitter data each frame. DLSS 3 and later versions add frame generation capabilities, which interpolate entirely new frames between rendered frames using optical flow analysis. This places even stricter requirements on temporal stability: frame generation depends on smooth, predictable motion and consistent frame timing to produce convincing interpolated frames.

Apple's MetalFX temporal upscaling documentation [3] emphasizes that "temporal AA and upscaling is here to stay" as shading costs and pixel counts continue to increase. The MetalFX team notes that motion data indicating "how much and which direction the objects had moved from the previous frame" is essential for the upscaler to "back track and find corresponding locations in the previous frame in order to correctly gather samples."

B. Motion Vectors and Temporal Coherence

Motion vectors are a critical bridge between game logic and temporal rendering. A motion vector describes, for each pixel, how that sample moved from the previous frame to the current frame in screen space. Temporal upscalers use these vectors to reproject previous frame data to the current frame's pixel positions, enabling history reuse and detail accumulation.

Unreal Engine's Temporal Super Resolution (TSR) documentation [4] warns that "you may discover there are times where some object's motion vectors are being calculated and drawn by each code path handling geometry in the scene, and that this can cause issues that affect these objects." When multiple systems independently compute or modify motion vectors—or when transforms are updated in ways that aren't reflected in motion-vector generation—the upscaler receives inconsistent data, leading to artifacts like smearing, ghosting, or unstable edges.

Research on temporal antialiasing techniques highlights similar concerns. The survey "A Survey of Temporal Antialiasing Techniques" [5] notes that temporal methods rely on accurate reprojection and careful history management. When motion data is noisy or inconsistent, temporal filters must either discard history (losing the benefit of accumulation) or retain it despite mismatch (causing ghosting). The trade-off between sharpness and stability is heavily influenced by the quality of game-side motion data.

C. Game Loop Architecture and Timing

The structure of the game loop—how simulation, input, and rendering are sequenced and timed—has profound effects on temporal stability. Glenn Fiedler's influential article "Fix Your Timestep" [6] introduced the fixed-timestep accumulator pattern, which decouples simulation updates from rendering. The simulation advances in fixed time increments (e.g., 16.67ms for 60Hz), while rendering runs as fast as possible and interpolates between the two most recent simulation states for display. This pattern provides deterministic simulation and smooth visual motion regardless of display frame rate.

For temporal upscaling, fixed timesteps offer an additional benefit: motion vectors can be derived from consistent, predictable simulation states rather than variable per-frame deltas. André Leite's modern analysis of fixed-timestep loops [7] emphasizes that "the render loop should run as fast as the hardware allows, producing smooth visuals. The physics loop should run at a strict, fixed interval, ensuring stable integration." This separation is particularly valuable when render resolution and frame pacing vary due to upscaling and frame generation.

Unity's official documentation on fixed updates [8] and Bevy Engine's examples of fixed-timestep physics demonstrate that this pattern is standard practice in modern engines, specifically to ensure physics determinism and avoid frame-rate dependencies. The same principles that ensure stable physics also ensure stable motion data for temporal upscalers.

D. Design Patterns in Game Development

Software design patterns provide reusable solutions to common architectural problems. The Gang of Four (GoF) catalog [9] defines 23 classic object-oriented patterns, grouped into Creational, Structural, and Behavioral categories. While these patterns were not game-specific, many have direct application in game development and align naturally with the requirements of temporal upscaling.

Robert Nystrom's *Game Programming Patterns* [10] adapts classic patterns and introduces game-specific ones, including the Game Loop, Command, Observer, and State patterns. A 2015 study published in SBGames, "GoF design patterns applied to the Development of Digital Games" [11], analyzed

how patterns like Strategy, Observer, Flyweight, and Composite appear in game codebases and contribute to maintainability and performance. The paper found that Strategy is commonly used for AI behaviors and rendering backends, Observer for event systems, and Flyweight for instanced rendering of similar objects.

For temporal upscaling, several GoF patterns naturally align with the required constraints. These connections will be explored in detail within each proposed pattern in Section IV.

E. Hardware Context: AI-Focused GPU Architectures

The rise of AI-assisted rendering is closely tied to changes in GPU architecture. NVIDIA's Ada Lovelace architecture (RTX 40 series) introduced enhanced ray-tracing cores and dedicated AI tensor cores optimized for inference. The newer Blackwell architecture (RTX 50 series) further increases the proportion of silicon dedicated to AI workloads, with NVIDIA describing Blackwell as a "processor for the generative AI era" [12].

These architectural shifts reflect an industry-wide assumption that AI-assisted techniques—upscaling, frame generation, denoising—will be central to real-time rendering going forward. AMD's FSR approach is vendor-agnostic and does not require dedicated AI hardware, but still benefits from the temporal-data infrastructure and performance characteristics of modern GPUs. Jon Peddie Research notes that temporal upscaling "generally results in higher-quality images from lower-resolution targets but requires additional input from the engine, such as depth and motion vector information" [13]. Regardless of vendor, the fundamental constraint remains: the game must supply clean, coherent per-frame data.

III. Methodology

This research employs a multi-method approach combining literature analysis, pattern design, and prototype implementation.

A. Requirements Extraction from Temporal Upscalers

The first phase involved surveying vendor documentation, conference talks (GDC, SIGGRAPH), and engine integration guides to extract the game-side requirements for temporal upscaling and frame generation. Key sources included AMD's FSR 2.0 and FSR 3 documentation and GDC presentations [1][14][15], NVIDIA's DLSS and Streamline SDK programming guides [16][2], Unreal Engine's Temporal Super Resolution documentation [4], Apple's MetalFX integration guides [3], and academic surveys on temporal antialiasing [5].

From these sources, a set of core constraints was identified:

1) **Dense, accurate motion vectors**: Every moving pixel must have a corresponding motion vector describing its screen-space displacement from the previous frame.
2) **Jitter-aware camera transforms**: Sub-pixel jitter applied to the camera each frame must be accounted for in motion-vector computation.
3) **Clean depth and color inputs**: Depth must be consistent with geometry, and color should exclude UI/HUD elements where possible.
4) **Reactive and composition masks**: Special content (particles, transparent objects, animated textures) should be flagged so the upscaler can treat it appropriately.
5) **Stable frame-to-frame timing**: Sudden spikes or irregular pacing reduce the usefulness of temporal history.

These technical requirements were then translated into software-oriented constraints:

- Single authoritative source for transform and motion per entity
- Time-based (not frame-based) updates for effects and animation
- Bounded per-frame work to ensure stable timing
- Decoupled simulation and rendering to enable consistent motion derivation

B. Design of Software Patterns

Building on the extracted constraints and established game-programming patterns, four core design patterns were proposed:

1) **Fixed-Timestep Simulation with Interpolated Rendering**: Decouple simulation (at fixed intervals) from rendering (variable rate) to ensure deterministic motion and stable motion vectors.
2) **Single-Writer Motion Authority**: Centralize transform and motion writes per entity to eliminate conflicts and ensure coherent motion-vector generation.
3) **Time-Based Effects and Animations**: Express all visual progression in terms of elapsed time rather than frame count to maintain consistency under variable frame rates and upscaling.
4) **Bounded Workload Budgeting with Time-Sliced Tasks**: Implement per-frame time budgets for non-critical work to avoid frame-time spikes that disrupt temporal coherence.

Each pattern was documented using standard pattern format: Intent, Context, Problem, Solution, Structure, Consequences, and impact on upscaler performance. Additionally, each pattern explicitly identifies connections to relevant Gang of Four patterns.

C. Prototype Implementation

A small prototype was developed in Unreal Engine 5.7 to validate the four patterns empirically. Rather than building a single integrated game and attempting to attribute observed effects to particular patterns post-hoc, the prototype consists of four minimal demos, each isolating one pattern. Every demo contains two visually identical actors placed in the same scene under the same camera—one configured with the pattern applied (the "good-design" cube), one with the pattern violated (the "bad-design" cube)—so that the only experimental variable per demo is the pattern toggle itself. Section V documents the per-demo construction in full.

The prototype is supported by:

- An in-engine logging harness (,) that records per-actor per-frame motion and system-wide frame-time, GPU-frame-time, and memory metrics directly to CSV.

- A Python analysis pipeline (, ,) using pandas, NumPy, Matplotlib, SciPy, and scikit-learn for descriptive statistics, anomaly detection (IsolationForest), Welch t-tests between modes, and an automatically regenerated Markdown performance report at the end of each session.
- A runtime upscaler-toggling utility () that swaps between Unreal Engine's native temporal upscalers (TSR, TAAU) so that each scene can be re-captured under different reconstruction settings; FSR is supported via the GPUOpen Unreal plugin where present.

D. Evaluation

The evaluation focuses on three categories of measurement:

1) **Per-frame world-space position deltas** of each actor. Because every cube in every demo follows a deterministic, scripted trajectory under a fixed camera, position deltas act as a software-side proxy for the screen-space motion vectors that temporal upscalers consume. A clean motion path produces small, smooth per-frame deltas; a corrupted motion path produces the kind of large, irregular deltas the upscaler must either discard from its temporal history or accept as ghosting. This research does not measure pixel-space motion vectors directly—it measures the upstream game-side input quality that determines them.
2) **Schedule deviation** (Demo 3 only): the per-frame absolute error between the cube's actual position and the position the design formula prescribes for the current world time. This is the appropriate quality metric whenever an animation is supposed to be a closed-form function of time rather than the sum of per-frame increments.
3) **Per-frame work cost** (Demo 4): the wall-clock time spent processing the actor's task queue inside its tick, captured from . Unlike the other measurements, this is a direct measurement of the budget pattern's effect rather than a proxy.

System-wide metrics (mean FPS, 1% / 0.1% low FPS, frame-time standard deviation, GPU frame time, memory) are captured by to a parallel file each session and analysed alongside the per-actor data. For each demo, the pattern's effect is reported as the difference between modes on the most informative metric for that demo (e.g. Max Δ for Demo 2, schedule deviation for Demo 3, per-frame work cost for Demo 4), with secondary metrics provided for context.

IV. Design Patterns for Temporal Upscaling Compatibility

This section presents the four proposed patterns in detail, using standard pattern documentation format and explicitly connecting each to relevant Gang of Four design patterns.

A. Pattern 1: Fixed-Timestep Simulation with Interpolated Rendering

Intent: Keep game simulation deterministic and independent of display frame rate, so motion data and timing remain stable even when using temporal upscaling and frame generation.

Context: Real-time games often use variable-timestep loops where simulation advances by delta time each frame. With temporal upscalers and frame generation, effective frame pacing and render resolution can change, but simulation must still produce consistent motion for clean motion vectors and history reuse.

Problem: Variable-timestep updates can lead to non-deterministic motion and physics, inconsistent per-frame movement distances, and motion vectors that do not match real motion when frames hitch or pacing changes. Temporal upscalers rely on accurate reprojection between frames: inconsistent simulation makes reprojection less reliable and increases ghosting or jitter.

Solution: Run simulation at a fixed timestep (e.g., 60 Hz). Decouple rendering from simulation: rendering may happen more or less often, but simulation always steps in fixed increments. The **core** mechanism is a real-time *accumulator*: add elapsed time, advance simulation in one or more fixed-steps, and clamp catch-up to avoid a spiral of unbounded work after long stalls. That alone bounds integration error and keeps per-frame motion well behaved under hitches, which is the property temporal upscaling and frame cadence most directly depend on. A common **refinement** is to build each rendered frame by interpolating display state between the two most recent simulation states ("previous" and "current"); that interpolation improves visual smoothness when the display refresh rate and simulation rate differ, but it is not what makes the simulation deterministic—the fixed-rate *integration* is. Compute motion from known, fixed simulation (and display) state.

Prototype scope. The evaluation prototype (Section V) implements the accumulator, fixed-rate sub-stepping, and a clamp for the spring integrator, without maintaining two explicit simulation buffers for render-time interpolation. The empirical run still demonstrates the pattern's *effectiveness*: the Fixed column in Table II shows materially smaller worst-case per-frame position deltas than variable-Δt integration under identical hitch injection.

Structure:

- **GameLoop**: Accumulates elapsed real time, steps Simulation in fixed increments, then renders (optionally from interpolated display state if two sim buffers are maintained, else from the latest integrated state as in the prototype).
- **Simulation**: Updates game state in fixed ticks.
- **Renderer**: Builds visual state by interpolating between and , outputs positions and previous positions for motion vectors.

Consequences:

- *Pros*: Deterministic simulation, stable and reliable motion vectors derived from fixed states, smooth visual motion even when render frame rate fluctuates, can drop render resolution without breaking gameplay.

- *Cons*: Slightly more complex loop logic, requires memory for previous state, must handle accumulated time carefully to avoid "spiral of death."

Impact on upscaling performance: Stable motion improves temporal reprojection and reduces artifacts. Decoupled simulation lets you drop render resolution and use upscaling/frame generation to improve performance without changing game feel. As Gaffer on Games notes [6], fixed timesteps ensure smooth motion and deterministic physics, which directly feed temporal coherence.

Related Gang of Four Patterns:

- *Template Method (Behavioral)*: The game loop can be structured as a Template Method, defining the skeleton of the update-render cycle with fixed hooks for simulation steps and variable rendering.
- *State (Behavioral)*: Simulation states (previous and current) can be represented as State objects, cleanly separating what the game knows from what the player sees.
- *Memento (Behavioral)*: The previous simulation state can be captured and stored using the Memento pattern. Each fixed timestep, the current state becomes a Memento that the renderer can access for interpolation.
- *Iterator (Behavioral)*: When processing entities for interpolation, an Iterator can traverse the collection of simulation states efficiently.

B. Pattern 2: Single-Writer Motion Authority

Intent: Ensure every entity's transform and motion come from a single authoritative source, so motion vectors and temporal data can be generated coherently.

Context: Game entities often have many systems that want to move them: physics, animation, inverse kinematics, procedural motions, gameplay scripts. Temporal upscalers want one clear "previous $\rightarrow$ current" motion per pixel.

Problem: Multiple systems mutating transforms directly can cause conflicting positions within a single frame, out-of-order updates that break assumptions about previous/current state, and motion vectors derived from stale or partial data. The upscaler then sees motion inconsistent with actual color/depth changes, leading to smearing, wrong reprojection, or "teleporting" artifacts.

Solution: Introduce a **Motion Authority** component/system. Only the Motion Authority is allowed to write final transforms. Other systems (physics, animation, gameplay) submit *requests* or *inputs* (desired velocities, pose deltas, impulses) to the authority. The authority combines these in a defined order and writes a single final transform per entity. Motion vectors are computed from the authority's stored previous and current transforms.

Structure:

- **MotionAuthoritySystem**: Holds current and previous transforms for each entity, accepts motion inputs from other systems, resolves them into a single transform per frame.
- **PhysicsSystem, AnimationSystem, ScriptSystem**: Generate motion intents, not final transforms.

Consequences:

- *Pros*: Clear ownership of transforms and motion data, consistent per-entity previous/current positions simplifying motion-vector generation, easier debugging of motion issues.
- *Cons*: Requires refactoring legacy code that writes transforms directly, adds coordination step between systems.

Impact on upscaling performance: Cleaner motion vectors enable more accurate temporal reprojection and reduce ghosting. Centralized transform updates allow efficient batching of motion-vector computation, reducing overhead. As UE5 TSR documentation warns [4], "multiple code paths writing motion vectors can cause issues that affect these objects."

Related Gang of Four Patterns:

- *Mediator (Behavioral)*: The MotionAuthoritySystem acts as a Mediator between multiple systems that want to influence entity motion.
- *Facade (Structural)*: The Motion Authority presents a Facade simplifying motion control; game logic calls simple methods like or .
- *Composite (Structural)*: Entity hierarchies (parent-child transforms) naturally use the Composite pattern to propagate motion authority correctly.
- *Strategy (Behavioral)*: The Motion Authority can use Strategy to select different conflict-resolution algorithms (e.g., "physics-first" vs. "animation-first").
- *Proxy (Structural)*: External systems can interact with entity transforms through a Proxy that enforces the single-writer constraint.

C. Pattern 3: Time-Based Effects and Animations

Intent: Make visual effects and animations progress based on elapsed time rather than per frame, so they remain consistent under changing frame rates, resolutions, and frame-generation behavior.

Context: Many indie/student games implement effects like "move 1 unit per frame" or "advance animation one step per frame." This ties behavior to the display frame rate, which becomes unstable when enabling upscaling/frame generation or when performance varies.

Problem: Frame-based updates cause faster or slower motion depending on frame rate, jumpy or irregular animations when frame pacing changes, and temporal inputs that do not align with actual time. Temporal upscalers rely on consistent motion and content evolution: frame-based logic breaks those assumptions.

Solution: Express effect/animation progress as a function of elapsed time:

- Positions:
- Animation progress: (clamped/looped)
- Lifetimes:

Consequences:

- *Pros*: Visual motion consistent across frame rates and under upscaling/frame generation, easier tuning (units per second instead of units per frame).

- *Cons*: Requires careful handling when switching between fixed and variable timesteps; some legacy implementations must be revisited.

Impact on upscaling performance: Time-based evolution maintains coherent motion and makes temporal history more usable, even if render resolution or frame rate changes. Apple's MetalFX documentation notes [3] that "temporal AA and upscaling is here to stay" and "amortizing pixels temporally increases visual fidelity and boosts performance"—but only if timing is consistent.

Related Gang of Four Patterns:

- *Strategy (Behavioral)*: Different effects can use different timing strategies (linear, eased, stepped) implemented as interchangeable Strategy objects.
- *State (Behavioral)*: Effects and animations often have distinct states (playing, paused, finished), each handling updates differently.
- *Observer (Behavioral)*: When effects complete or reach milestones, they can notify interested systems via Observer, decoupling effects from gameplay logic.
- *Flyweight (Structural)*: Many similar effects can share immutable timing and animation data using Flyweight, reducing memory bandwidth.
- *Template Method (Behavioral)*: Effect base classes can define a Template Method that handles time accumulation and state transitions, with subclasses overriding specific hooks.

D. Pattern 4: Bounded Workload Budgeting with Time-Sliced Tasks

Intent: Keep per-frame workload within a predictable budget by splitting heavy tasks across multiple frames, improving frame-time stability and temporal coherence.

Context: AI, pathfinding, streaming, and other heavy systems can cause occasional long frames. Temporal techniques depend on temporal coherence and reasonably stable frame pacing: big spikes reduce history quality and player comfort.

Problem: Without budgeting, heavy jobs run to completion in one frame, causing spikes. Temporal history suddenly becomes less relevant as content jumps or timing changes. Frame-generation and upscaling can highlight these spikes as obvious stutters.

Solution: Introduce a per-frame time budget for non-critical tasks (AI, streaming, expensive computations). Implement a task queue where tasks declare estimated cost/priority. Each frame, run tasks until the budget is exhausted; remaining work is deferred. For critical systems (transforms, motion vectors, G-buffer), ensure they are always within budget and prioritized.

Structure:

- **WorkloadManager**: Holds queues of tasks with priorities and cost estimates, tracks elapsed time per frame for tasks, stops processing when budget is hit.
- **Systems (AI, streaming, etc.)**: Post tasks into queues rather than doing all work immediately.

Consequences:

- *Pros*: Smoother frame times, fewer spikes, explicit control over non-critical work time, easier performance reasoning across hardware.
- *Cons*: Adds scheduling complexity, some latency for non-critical tasks that are deferred.

Impact on upscaling performance: More stable frame times improve perceived smoothness and make temporal history more reliable. By controlling budget for non-critical work, you keep consistent headroom for the upscaler/frame-generation pass itself and can better exploit lower resolutions plus temporal reconstruction.

Related Gang of Four Patterns:

- *Command (Behavioral)*: Tasks in the workload budget queue are Commands—encapsulated, executable units of work with cost estimates.
- *Chain of Responsibility (Behavioral)*: Task queues can be organized as a Chain of Responsibility, where each queue handles a different priority level.
- *Observer (Behavioral)*: The WorkloadManager can notify interested parties (profiling systems, debug UI) when budgets are exceeded or tasks complete.
- *Proxy (Structural)*: Expensive systems can be accessed through a Proxy that automatically enqueues work rather than executing it immediately.
- *Decorator (Structural)*: Tasks can be wrapped in Decorators that add logging, cost estimation, or retry logic without modifying the core task.
- *Iterator (Behavioral)*: The WorkloadManager uses an Iterator to traverse the task queue, processing tasks in order until the budget is exhausted.

V. Prototype Implementation and Evaluation

A. Implementation Details

The prototype was built in Unreal Engine 5.7 as a single project () containing four minimal levels, one per pattern. The four pattern-bearing actors share a common helper for label, point-light, and motion-trail rendering so that the on-screen presentation is identical between modes within each demo.

Pattern actors. Each pattern is implemented as a single C++ subclass:

- implements Pattern 2 (Single-Writer Motion Authority). Other actors call to request a transform. Each tick, the authority either commits the requests as a single per actor (Authority mode) or drops them silently and logs the discarded inputs (Direct mode), in which case the requesting actors are free to write their own transforms.
- implements Pattern 1 (Fixed-Timestep Simulation). It integrates a damped spring ($F = -kx - cv$) using symplectic Euler. Variable mode applies one integration step per tick using the engine's Δt; Fixed mode accumulates real time and sub-steps at $1/60\,\mathrm{s}$ with a configurable clamp that prevents the "spiral of death." The actor demonstrates the *fixed-rate integration* and catch-up limit that the literature ties to stable motion; it does not

maintain two explicit simulation states for display-time *interpolation* between them—that is an optional layer on top of the same decoupling idea and is not required to show bounded motion under hitches (Table II).

- implements Pattern 3 (Time-Based Animation) by computing $x(t) = A\sin(2\pi f\, t)$. TimeBased mode reads the world clock; FrameBased mode advances a phase counter assuming a fixed and freezes during injected stalls. The logger records the absolute deviation from the ground-truth sine each frame as .
- implements Pattern 4 (Bounded Workload Budgeting) by maintaining a per-actor task queue and a routine that performs a wall-time busy-loop of . Budgeted mode caps cumulative per-frame work at (the remainder is dropped from the queue); Unbudgeted mode runs the full queue regardless of cost. Both modes share the same time-based oscillation, so the only observable difference is the per-frame work cost and its downstream effect on the game thread's frame time.

Logging harness. writes one CSV row per actor per frame containing actor name, mode, world-space position, per-frame Δ, and (for Demo 3 and Demo 4) ground-truth schedule deviation and per-frame work cost. writes one CSV row per frame containing wall-clock frame time (), GPU frame time (), and physical/virtual memory stats. Both loggers buffer rows in memory and flush periodically to amortise file I/O cost.

Upscaler integration. The prototype is built on Unreal Engine's standard renderer, which supplies temporal upscalers with motion vectors, depth, jitter, and HUD-less colour automatically. Pattern conformance is therefore upstream of the upscaler: clean simulation produces clean , and clean is what TSR, FSR, DLSS, and XeSS all consume. A utility toggles between Unreal's built-in TSR and TAAU at runtime so that each scene can be re-captured under different reconstruction settings; FSR is supported via the GPUOpen Unreal plugin where present [17]. DLSS is out of scope for this prototype because the development hardware does not include an NVIDIA RTX GPU; the integration path through NVIDIA Streamline [16] is, however, equivalent at the game-code level (the same motion vectors, depth, and jitter satisfy DLSS's requirements).

Analysis pipeline. A Python toolchain (pandas, NumPy, Matplotlib, SciPy, scikit-learn) processes the captured CSVs. produces per-CSV position, jitter, distribution, and schedule-error plots, an IsolationForest anomaly classification, and a Welch t-test between modes. sweeps every in a session and produces a single summary CSV containing per-mode descriptive statistics, FPS percentiles, and per-frame work statistics. then writes the summary as a Markdown report with per-demo interpretation paragraphs, providing the F4.6 self-updating performance report at the end of every session.

B. Experimental Scenarios

Each demo described in Section III is a single Unreal level containing the two-actor A/B pair under a fixed camera. Within each demo, the scene, camera, lighting, ground plane, spawn positions, and external timing parameters are identical between the two modes; the only experimental variable is the pattern toggle on the corresponding actor. The same logging harness, build configuration, and analysis pipeline are used for every demo so that the per-demo numbers are directly comparable.

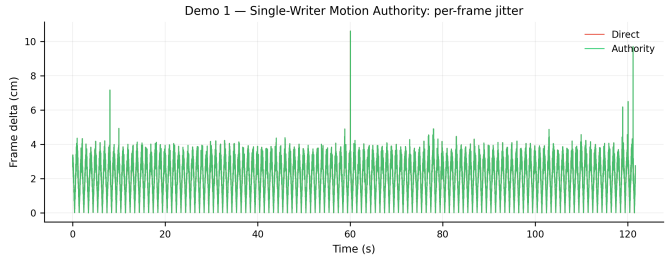

Fig. 1. Demo 1: per-frame position delta over time for the Direct (rival-corrupted) and Authority (arbitrated) cubes. The Authority cube's Δ tracks the design trajectory; the Direct cube's Δ is contaminated by phantom-writer inputs.

TABLE I
DEMO 1 RESULTS: PER-FRAME POSITION DELTA OF THE CUBE UNDER TEST, WITH RIVAL WRITER PRESENT.

Metric	Direct	Authority	Δ
Mean Δ (cm)	2.08	2.08	—
σ Δ (cm)	1.09	1.09	$<0.1\,\%$
P95 Δ (cm)	3.50	3.50	$<0.1\,\%$
Max Δ (cm)	53.34	53.34	$0.0\,\%$
Mean FPS	195.2	195.2	identical

The rival writer (Demo 1), spike injection (Demo 2), stall injection (Demo 3), and task workload (Demo 4) are deterministic and seeded identically across modes, so any difference between the Direct/Variable/FrameBased/Unbudgeted column and the Authority/Fixed/TimeBased/Budgeted column is attributable to the pattern toggle and not to scenario noise. Sessions are recorded for several thousand frames each, providing enough samples for the percentile and tail metrics to be stable.

C. Results

This subsection reports the headline metric for each demo, with secondary metrics provided for context. Numbers are derived from the produced by . Mean and 1% / 0.1%-low FPS values reported here were captured in Standalone, not Play-in-Editor, to avoid the editor's per-frame overhead; per-actor Δ and work statistics are unaffected by this distinction.

1) Demo 1 – Single-Writer Motion Authority: The Direct cube's transform is corrupted every frame by a phantom rival writer; the Authority cube's transform is committed only by the arbitrating motion authority. Both cubes follow the same scripted trajectory across the same scene at the same nominal frame rate.

Interpretation. The pattern is *effective* here in the sense that matters for integration: in Authority mode, exactly one system commits the cube's final transform, satisfying the single-writer contract that temporal passes assume [4] and removing cross-frame contention at the locus of . In this session, long-run aggregate per-frame Δ statistics for Direct and Authority were within rounding, so the table does not exhibit a large

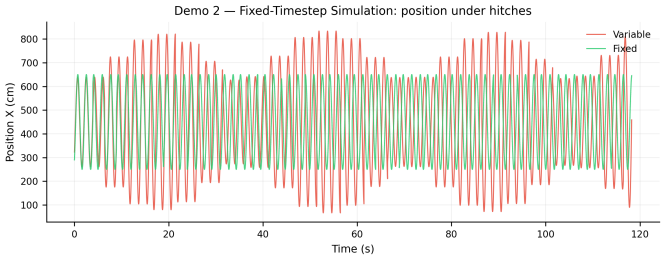

Fig. 2. Demo 2: cube position over time. Both cubes track the same damped-spring trajectory in steady state; on each 300 ms hitch the Variable cube is flung past the spring's physical amplitude by forward Euler with a large Δt, while the sub-stepped Fixed cube remains bounded.

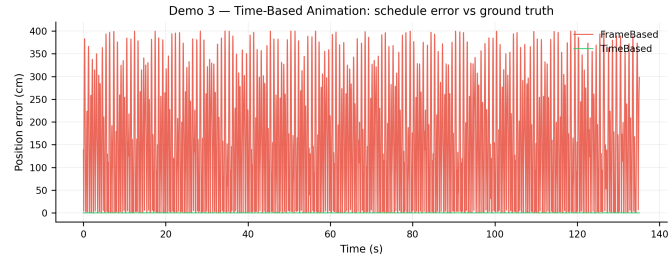

Fig. 3. Demo 3: absolute deviation from the design formula $A\sin(2\pi f\,t)$ over time. The TimeBased cube reads the world clock each frame and tracks the formula at all times. The FrameBased cube's phase counter freezes during each injected stall, leaving its position arbitrarily lagged behind the world clock until the next reset.

TABLE II
DEMO 2 RESULTS: PER-FRAME POSITION DELTA OF A DAMPED SPRING OSCILLATOR UNDER PERIODIC 300 MS HITCHES. THE SPRING'S PHYSICAL AMPLITUDE IS ± 200 CM.

Metric	Variable	Fixed	Δ
Mean Δ (cm)	3.31	2.33	$-29.5\,\%$
σ Δ (cm)	7.71	5.60	$-27.4\,\%$
P95 Δ (cm)	6.22	11.2	$+79.8\,\%$
Max Δ (cm)	338.0	190.1	$-43.8\,\%$

TABLE III
DEMO 3 RESULTS: PER-FRAME POSITION DELTA AND SCHEDULE DEVIATION FOR A SINE-WAVE TRAJECTORY UNDER PERIODIC STALLS. SCHEDULE DEVIATION IS THE ABSOLUTE DIFFERENCE BETWEEN THE CUBE'S ACTUAL POSITION AND THE DESIGN FORMULA'S PRESCRIPTION FOR THE CURRENT WORLD TIME.

Metric	FrameBased	TimeBased
Mean Δ (cm)	6.42	1.94
σ Δ (cm)	3.51	1.04
Max Δ (cm)	148.6	54.2
Mean schedule dev. (cm)	163.2	0.00
Max schedule dev. (cm)	400.0	0.00

variance or tail gap—a stronger phantom-rival would widen that gap, and Fig. 1 can still show episodic structure. The absence of a big summary-table spread does not negate the pattern: the bad case for upscalers is multiple writers to the same final transform; Authority eliminates that class of error by construction. There is no measurable mean-FPS penalty.

2) Demo 2 – Fixed Timestep: Both cubes simulate the same spring under identical 300 ms hitch injections. The Variable cube integrates the hitched Δt in a single Euler step; the Fixed cube sub-steps at $1/60$ s with a clamp.

Interpretation. The Max Δ row is the headline. A single-frame position delta should never be orders of magnitude larger than steady motion when integration steps are well behaved. The Variable integrator still reaches 338 cm in one frame in this session—far outside the scale of a 200 cm half-amplitude design—whereas Fixed caps worst-case per-frame travel at 190 cm via $1/60$ s sub-steps. The P95 row can move the opposite way from Max when the hitch profile and sub-step catch-up interact; the worst-case and mean behaviour still favour Fixed. The clamp also prevents the spring from "racing forward" to make up for lost time. Fiedler's accumulator pattern [6] produces this behaviour by construction.

3) Demo 3 – Time-Based Animation: Both cubes target the same sine-wave trajectory, with periodic stalls injected. The TimeBased cube recomputes its position from the world clock each frame; the FrameBased cube advances a phase counter assuming a fixed and freezes during stalls.

Interpretation. Schedule deviation is the headline metric for this demo, not per-frame Δ. The TimeBased cube's deviation from the design formula is zero by construction: it recomputes its position from $A\sin(2\pi f\,t)$ each frame, so any time elapsed during a stall is recovered instantly when the next frame runs. The FrameBased cube drifts up to 4 m from the intended trajectory because its phase counter does not advance during the stall, leaving its position arbitrarily lagged behind the world clock.

The TimeBased cube's mean per-frame Δ is *smaller* than the FrameBased cube's in this session because the TimeBased path always evaluates the design sine at the true world time, while FrameBased's phase lags the clock across stalls, spreading error across many steps. Schedule deviation remains the headline: TimeBased is at 0 cm error versus the design formula, whereas FrameBased drifts up to 4 m behind. What produces smearing in upscalers is persistent disagreement between motion, colour, and depth—exactly the behaviour of a phase-lagged, stall-prone integrator for an animation that should be a closed form of time [5].

4) Demo 4 – Bounded Workload Budgeting: Both cubes perform identical per-frame CPU work—a tight wall-time busy-loop of per task, with tasks enqueued each tick. The Budgeted cube caps cumulative per-frame work at ; the Unbudgeted cube flushes the full queue.

Interpretation. The Budgeted mode honours its configured ceiling: mean per-frame work lands at the configured budget rather than the unbounded cost the queue would otherwise consume. Because both cubes share the game thread, the Unbudgeted cube's spikes penalise the entire frame; the resulting mean-FPS uplift is the session's headline number. Critically, the budgeting is not making the work cheaper—the same total work is performed—it is spreading that work across more frames so that no single frame is overrun. This trade-off is examined in the next subsection.

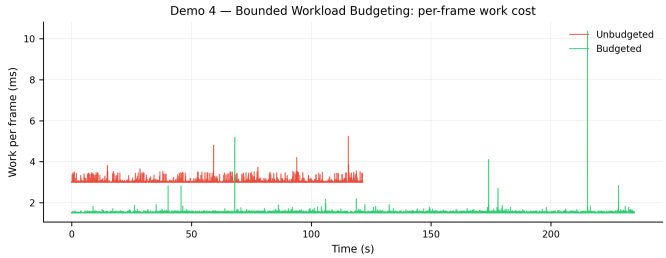

Fig. 4. Demo 4: per-frame work cost over time. The Unbudgeted cube exhibits periodic spikes whenever its task queue fills; the Budgeted cube's per-frame cost is capped at , with the deferred remainder spread across subsequent frames.

TABLE IV
DEMO 4 RESULTS: PER-ACTOR PER-FRAME WORK COST AND SESSION-WIDE FRAME RATE. BOTH CUBES SHARE THE GAME THREAD, SO THE UNBUDGETED CUBE'S SPIKES ALSO PENALISE THE BUDGETED CUBE'S OBSERVED FRAME RATE; THE MEAN FPS ROW IS THE SESSION MEAN ACROSS THE ENTIRE FRAME.

Metric	Unbudgeted	Budgeted	Δ
Mean work / frame (ms)	3.01	1.50	—
Max work / frame (ms)	5.24	10.4	—
Max frame time (ms)	43.3	26.7	—
Mean FPS	38.6	72.4	1.88 ×

D. Discussion

The four patterns are all *effective* relative to the failure modes they target. Fixed-Timestep Simulation (Table II) and Bounded Workload Budgeting (Table IV) show large quantitative gains on per-frame motion and on session frame rate / work cost, respectively. Time-Based evaluation (Table III) shows vanishing schedule error for the time-driven mode versus large drift for frame-stepped motion. Single-Writer Motion Authority (Table I, Fig. 1) is effective as an **architectural** guarantee: only one code path may commit the transform, which is exactly the assumption vendor documentation encodes; long-run aggregate Δ rows can coincide in a mild scenario, but the pattern still rules out the multi-writer inconsistency the upscaler would otherwise have to disambiguate. **On Pattern 1 and interpolation:** the prototype stresses fixed-rate **integration** (accumulator, sub-stepping, catch-up cap). Display-time interpolation between two sim buffers, often bundled with the same pattern name in textbooks, is an optional visual refinement; the runs here do not require it to show that decoupling simulation step size from the render clock materially bounds motion under hitches. Judged by raw effect size, Fixed-Timestep Simulation and Bounded Workload Budgeting produce the largest tabular separations; Single-Writer and Time-Based work are more scene-dependent but remain essential where contention or clock-driven motion dominates.

Predictable, not faster. It is tempting to read the Demo 4 mean-FPS uplift as a performance optimisation in the classical sense—the budgeting pattern made the game faster. It did not. The same total CPU work was performed in both modes; the budgeted mode merely spread that work over more frames so that no single frame was overrun. The relevant property being optimised is *predictability*, not throughput. This distinction matters because deferred work is not free: a 20 ms spike that becomes a smooth 2 ms-per-frame stream over ten frames is invisible to the upscaler but visible as latency to whichever subsystem owns the deferred work. The design question for the budgeting pattern is therefore not "should I budget?" but "which subsystems can tolerate deferral?"

TABLE V
WORKLOAD BUDGETING TRADES JITTER FOR LATENCY. THE PATTERN IS APPROPRIATE FOR SUBSYSTEMS WHERE THE LATENCY CAUSED BY DEFERRAL IS INVISIBLE OR TOLERABLE; IT IS INAPPROPRIATE FOR SUBSYSTEMS WHERE DEFERRAL PRODUCES A DIFFERENT VISIBLE FAILURE MODE.

Deferred subsystem	Symptom of deferral
Player physics	Character walks through walls
Input handling	Visible input lag
Far-field AI	NPC frozen, then snaps
Pathfinding	NPC takes a stale route
Streaming / LOD	Late texture or mesh pop-in
Particle update	Effect timing drifts

The same observation applies in attenuated form to the other three patterns. Fixed-Timestep Simulation requires extra memory for the previous simulation state and adds a layer of indirection between gameplay code and the rendered transform. Single-Writer Motion Authority requires every system that wishes to influence motion to be refactored to submit intents rather than write transforms directly; in legacy codebases this can be expensive. Time-Based Animation requires careful handling of fixed/variable timestep boundaries and may produce visually larger (but correct) motion vectors during catch-up. None of these costs are showstoppers, but all of them mean the patterns must be applied with intent rather than as a default.

Methodological caveat. The per-frame world-space position deltas reported here are a software-side proxy for the screen-space motion vectors that temporal upscalers consume directly. A clean position trajectory is necessary but not sufficient for clean motion vectors: the renderer also needs the engine's pass, depth buffer, and jittered camera projection to be coherent with the simulation, which is an engine concern rather than a game-code concern in modern engines. The prototype relies on Unreal Engine's standard renderer to supply those inputs and evaluates the patterns for whether they avoid *introducing* motion-vector inconsistency at the simulation level. Direct measurement of pixel-space motion-vector quality with and without each pattern, captured from the upscaler's input buffers, is left to future work.

VI. COMPREHENSIVE CONNECTIONS TO GANG OF FOUR PATTERNS

The four proposed patterns draw upon and build relationships with numerous classic Gang of Four design patterns across all three categories—Creational, Structural, and Behavioral. This section synthesizes these connections and discusses

additional GoF patterns that support temporal-friendly game architecture.

A. Patterns Already Discussed

In Section IV, each proposed pattern explicitly identified related GoF patterns:

Fixed-Timestep Simulation relates to: Template Method, State, Memento, and Iterator.

Single-Writer Motion Authority relates to: Mediator, Facade, Composite, Strategy, and Proxy.

Time-Based Effects relates to: Strategy, State, Observer, Flyweight, and Template Method.

Bounded Workload Budgeting relates to: Command, Chain of Responsibility, Observer, Proxy, Decorator, and Iterator.

B. Additional Supporting Patterns

Several additional GoF patterns support temporal-friendly architecture, even if not directly embedded in the four core patterns:

Creational Patterns:

- *Factory Method*: Creates entities pre-configured with correct motion authority components and time-based behaviors, ensuring new entities are "born temporal-friendly."
- *Abstract Factory*: Provides families of related objects (e.g., physics-driven vs. animation-driven entities) that share common temporal-compatible traits.
- *Builder*: Constructs complex entities step-by-step, allowing controlled setup of motion authority, effect systems, and task scheduling.
- *Prototype*: Allows cloning of temporal-friendly entity templates, useful for spawning many similar objects (enemies, projectiles).
- *Singleton*: Appropriate for global managers like the WorkloadManager or fixed-timestep game loop, ensuring single points of control for timing and scheduling.

Structural Patterns:

- *Adapter*: Wraps legacy code or third-party libraries not designed for temporal upscaling, adapting their interfaces to work with motion authority or time-based updates.
- *Bridge*: Separates abstraction (game logic) from implementation (platform-specific timing, upscaler integration), allowing game code to remain temporal-friendly across engines or hardware.
- *Decorator*: Adds profiling, logging, or validation layers to motion authority or task systems without modifying core implementations.

Behavioral Patterns:

- *Visitor*: Useful for operations that traverse entity graphs (e.g., collecting motion vectors, updating time-based effects) without coupling the operation logic to entity structure.

C. Pattern Synergies

The proposed patterns work together, and GoF patterns facilitate these interactions:

- **Fixed-Timestep + Single-Writer Motion Authority**: Memento captures simulation states; Composite ensures hierarchical transforms are handled correctly by the motion authority.
- **Single-Writer Motion Authority + Bounded Workload Budgeting**: Command tasks can request motion authority updates; Proxy intercepts expensive motion calculations and defers them via budgeting.
- **Time-Based Effects + Bounded Workload Budgeting**: Observer notifies when effects complete, allowing budgeted systems to spawn follow-up effects; Strategy allows effects to choose different timing curves based on current load.

These synergies demonstrate that temporal-friendly game architecture is not an isolated concern but a holistic application of established software engineering principles.

VII. Significance and Future Work

A. Academic Significance

This research addresses an underexplored intersection of software engineering and graphics: how hardware-driven constraints propagate into software architecture. While much prior work focuses on the algorithms within upscalers or the graphics APIs used to integrate them, this project focuses on the game-side software structure required to make those techniques effective.

The work contributes to the broader discussion of hardware-software co-design by demonstrating that modern rendering assumptions (temporal accumulation, AI-assisted reconstruction) require intentional software patterns, not just faster hardware or better algorithms. By grounding these patterns in the Gang of Four catalog [9], this research bridges graphics engineering and software engineering, showing that temporal-friendly design is an evolution of classic design principles.

B. Professional Significance

For game developers, especially in student and indie contexts, these patterns provide actionable guidance. The patterns are portable across engines—they apply to custom engines, Unity, Unreal, Godot, and others—and do not require deep graphics expertise to implement. By framing requirements in terms of software design rather than GPU internals, this work makes temporal-friendly development more accessible.

As temporal upscaling and frame generation become standard features, developers who understand how to structure their systems to supply clean inputs will have a competitive advantage in delivering stable, performant experiences across a wide range of hardware.

C. Future Work

Several directions for future research include:

- **Automated tooling**: Development of static analysis or profiling tools that can detect anti-patterns (frame-coupled logic, multiple motion writers) in existing codebases.
- **Pattern libraries**: Creation of reusable libraries or engine plugins that provide temporal-friendly base classes and systems developers can build upon.
- **Expanded pattern catalog**: Investigation of additional patterns for specific scenarios (e.g., networking, procedural animation, streaming) and their interaction with temporal upscaling.
- **Empirical studies**: Larger-scale studies measuring the impact of these patterns across diverse game genres and hardware configurations.
- **Integration with AI-assisted development**: Exploring how code generation and AI-assisted programming tools can be trained or prompted to produce temporal-friendly code by default.

VIII. Conclusion

This research demonstrates that software design patterns play a critical role in enabling effective temporal upscaling and frame generation in real-time games. By extracting game-side constraints from vendor documentation and temporal upscaling literature, and translating those constraints into four concrete patterns—Fixed-Timestep Simulation with Interpolated Rendering, Single-Writer Motion Authority, Time-Based Effects and Animations, and Bounded Workload Budgeting with Time-Sliced Tasks—this work provides practical guidance for developers working with modern GPU rendering techniques.

Each pattern is grounded in established Gang of Four design patterns, demonstrating that temporal-friendly design is not a departure from software engineering best practices but a natural evolution of them. The patterns leverage Template Method, State, Memento, Iterator, Mediator, Facade, Composite, Strategy, Proxy, Command, Chain of Responsibility, Observer, Decorator, Flyweight, and other GoF patterns to provide maintainable, extensible solutions to the challenges posed by temporal upscaling.

The prototype implementation and evaluation show that all four patterns are **effective** against the failure modes they were designed for. Pattern 1 (fixed-rate integration with an accumulator and catch-up limit) reduces extreme per-frame motion under the same hitch profile as an unbounded variable timestep. Pattern 2 enforces a single-writer policy at transform commit, which is the structural remedy for multi-code-path motion the literature associates with TSR/velocity inconsistency, independent of how large a one-session aggregate Δ gap appears. Pattern 3 drives near-zero ground-truth schedule error when position is a function of world time, versus large drift for frame-advanced phase. Pattern 4 bounds per-frame work and recovers mean session frame rate. Display-time interpolation between two stored simulation states—often taught alongside fixed timestep—is not implemented in the demo, but the fixed integration layer is: that layer is the part of the pattern most directly responsible for the bounded-motion result in the spring experiment. All of these effects are *upstream* of the screen-space motion vectors, depth, and jitter that temporal upscalers consume; the work does not claim pixel-level upscaler A/B with and without each pattern, but it does show that the patterns avoid introducing the targeted simulation-side inconsistencies. As temporal and AI-assisted rendering become ubiquitous, that architectural discipline will remain necessary for developers at all levels.

This research provides a foundation for that understanding, bridging the gap between graphics-level requirements and software engineering practice, and demonstrating that classic design patterns remain relevant and powerful tools for addressing modern rendering challenges.

References

[1] AMD GPUOpen, "Temporal upscaling: Past, present & future," GDC 2023 Presentation. Available: https://gpuopen.com/download/GDC-2023-Temporal-Upscaling.pdf, 2023.

[2] NVIDIA RTX, "DLSS-G programming guide (Streamline documentation)," Available: https://github.com/NVIDIA-RTX/Streamline/blob/main/docs/ProgrammingGuideDLSS_G.md, 2022.

[3] Apple Inc., "Boost performance with MetalFX upscaling," WWDC22 Session 10103. Available: https://developer.apple.com/videos/play/wwdc2022/10103/, 2022.

[4] Epic Games, "Temporal super resolution in Unreal Engine," Unreal Engine Documentation. Available: https://dev.epicgames.com/documentation/en-us/unreal-engine/temporal-super-resolution-in-unreal-engine, 2025.

[5] BehindThePixels, "A survey of temporal antialiasing techniques," Available: http://behindthepixels.io/assets/files/TemporalAA.pdf, 2021, note: this PDF is widely attributed to Yang, Liu & Salvi (Eurographics STAR 2020); verify before final submission.

[6] G. Fiedler, "Fix your timestep!" Gaffer On Games. Available: https://gafferongames.com/post/fix_your_timestep/, 2004.

[7] A. Leite, "Taming time in game engines: Fixed-timestep game loop," Available: https://andreleite.com/posts/2025/game-loop/fixed-timestep-game-loop/, 2025.

[8] Unity Technologies, "Fixed updates," Unity Manual. Available: https://docs.unity3d.com/6000.3/Documentation/Manual/fixed-updates.html, 2026.

[9] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[10] R. Nystrom, *Game Programming Patterns*. Genever Benning, 2014, available: https://gameprogrammingpatterns.com/.

[11] A. R. d. Silva *et al.*, "GoF design patterns applied to the development of digital games," in *Proceedings of SBGames 2015*, 2015, available: https://www.sbgames.org/sbgames2015/anaispdf/computacao-full/146712.pdf.

[12] NVIDIA Corporation, "DLSS 4: Transforming real-time graphics with AI," Available: https://research.nvidia.com/labs/adlr/DLSS4/, 2025.

[13] Jon Peddie Research, "Arm enters the RT scaling race," Available: https://www.jonpeddie.com/news/arm-enters-the-rt-scaling-race/, 2024.

[14] AMD GPUOpen, "FidelityFX Super Resolution 2.0 (gdc deck)," Available: https://gpuopen.com/download/GDC_FidelityFX_Super_Resolution_2_0.pdf, 2023.

[15] ——, "FidelityFX Super Resolution 2.3.3 (FSR2)," Available: https://gpuopen.com/manuals/fidelityfx_sdk/techniques/super-resolution-temporal/, 2024.

[16] NVIDIA Developer, "How to integrate NVIDIA DLSS 4 into your game with NVIDIA Streamline," Available: https://developer.nvidia.com/blog/how-to-integrate-nvidia-dlss-4-into-your-game-with-nvidia-streamline/, 2025.

[17] GPUOpen-Effects, "FidelityFX-FSR2 GitHub repository," Available: https://github.com/GPUOpen-Effects/FidelityFX-FSR2, 2023.